

Silent Attractors: Adversarial Task Design for Detecting Latent Reasoning Failures in Frontier Language Models

Polex Ouma Otieno¹

¹Independent Researcher / Smartlinks Ltd

August 8, 2026

Abstract

Standard benchmarks increasingly saturate against frontier language models, yet models continue to fail in production on tasks that require one non-obvious reasoning step diverging from a trained professional reflex. We introduce a task design methodology for constructing *adversarial evaluation tasks* that isolate this failure mode, which we term a **silent attractor**: a plausible, confidently stated answer produced by a common heuristic or memorized pattern that is not supported by the specific facts of the problem. Each task is validated against two references before inclusion: an oracle solution that must pass, and a naive/reflex solution constructed from the most common professional shortcut, which must fail. We describe a nine-family taxonomy of failure mechanisms observed across accepted tasks spanning financial accounting, pension cost restatement, and distributed systems domains, and report pass rates of current frontier models against a pilot task set. We release our task construction methodology, a reproducible evaluation harness, and the pilot task set to support further adversarial benchmark construction.

1 Introduction

Large language models are increasingly deployed in domains where a wrong-but-confident answer carries real financial or operational cost: accounting close processes, engineering sign-off, and regulatory reporting among them. Aggregate benchmark accuracy, however, is a poor proxy for reliability in these settings, because a model can score well on average while reliably failing on the specific subset of problems where a memorized professional reflex diverges from the fact pattern at hand.

This paper describes a task construction methodology, developed and iterated across an adversarial benchmark contribution platform, for deliberately isolating this failure mode. We call it a **silent attractor**: the model converges on an answer that would be correct under the common case, but the specific problem instance has been constructed so that one fact — present in the prompt, never flagged as unusual — changes the correct derivation path. The model is not being tricked with irrelevant information or adversarial phrasing; every fact given is necessary and sufficient to solve the problem correctly. The failure is purely a reasoning failure: applying the high-frequency pattern instead of deriving the answer from the specific facts given.

Contributions. (1) A task construction and validation methodology — oracle-passes / naive-reflex-fails — for building silent-attractor tasks. (2) A nine-family taxonomy of the reasoning mechanisms that produce this failure mode. (3) A reproducible evaluation harness (task loader, multi-provider model runner, automatic verifier, and statistical reporting with Wilson score confidence intervals) release alongside a pilot task set. (4) Pilot results across current frontier models on financial accounting tasks constructed with this methodology.

2 Related Work

Adversarial and contamination-resistant benchmark construction has grown alongside concerns that static benchmarks saturate and leak into training data. Work on LLM-judge reliability has shown that automated evaluators themselves exhibit systematic biases, including overconfidence and reduced sensitivity to expert-identified weaknesses compared to human raters, motivating evaluation designs where the correct answer is verifiable by direct computation rather than judged by another model. Separately, benchmark work on agentic and multi-turn settings has shown that models are sensitive to distributional shift away from the common case, including under peer pressure from unreliable collaborators. Our work differs in scope: we target single-turn, professionally grounded quantitative tasks in which the shift away from the common case is entirely contained in the stated facts, with no adversarial framing, multi-agent interaction, or distributional shift required.

3 Task Construction Methodology

Each task begins from a real professional workflow (e.g., a pension cost restatement, a foreign currency consolidation) and is built through the following stages.

1. **Identify the reflex.** Identify the single most common shortcut, heuristic, or memorized formula a trained professional would reach for given the surface features of the problem.
2. **Identify the crux.** Identify one fact, derivable without any information beyond what is given, that changes the correct derivation path away from that reflex.
3. **Construct the oracle.** Write a reference solution that correctly incorporates the crux.
4. **Construct the naive baseline.** Write a second solution that applies the identified reflex and ignores the crux.
5. **Validate discrimination.** Confirm the oracle answer and naive answer differ by a margin well outside the task’s stated tolerance, so the task can actually discriminate between correct and merely-plausible reasoning.
6. **Author the instruction.** Write the task prompt so that every fact needed for the oracle solution is present, unflagged, and stated plainly, with no hint that a shortcut would be incorrect.
7. **Assemble source artifacts.** Where the underlying workflow naturally spans multiple documents (a data extract, a spreadsheet, a policy memo or plan document), construct each as a separate artifact rather than a single flattened prompt, so the task requires cross-referencing rather than single-document reading comprehension.

This mirrors a validated production workflow: every task in the pilot set was built and reviewed through a formal proposal and multi-stage technical review pipeline before inclusion, including an automated check that the oracle solution passes and the naive solution fails.

3.1 Failure mechanism taxonomy

Across accepted tasks, we identify nine recurring mechanisms by which a silent attractor is constructed. We name each mechanism descriptively rather than by an arbitrary label, since the name itself is part of the contribution:

Reflex override A trained professional reflex produces the wrong answer; correct reasoning requires one non-obvious derivation step the data supports but never announces.

Heuristic substitution A shortcut resembles the correct method closely enough to look right while silently diverging from it.

Distractor injection A distractor resource or datum invites misuse if not filtered out.

Evidence synthesis gap The answer must be inferred from partial evidence rather than read off directly.

Assumption violation An assumption that normally holds is violated in this specific instance.

Weak-signal fusion The failure only becomes visible once multiple individually weak signals are combined.

Exhaustive coverage failure Many sub-parts must all be correct simultaneously for the task to be scored as passed.

Temporal coupling Order-of-operations or path dependence changes the correct answer.

Staleness error A stale or superseded value is used where the value as of a specific date is required.

The two pilot tasks released with this paper (Section 5) instantiate temporal coupling and assumption violation respectively.

4 Evaluation Harness

We release a lightweight, provider-agnostic evaluation harness alongside this paper (see the accompanying code repository):

- **Task loader** parses a task directory (`task.toml` + `instruction.md`) into a uniform schema, so task exports from the source platform require minimal reshaping.
- **Model runner** sends each task’s instruction to one or more configured model providers and stores the raw completion. Anthropic and OpenAI clients are provided; adding a provider requires implementing a single `complete(prompt)` method.
- **Verifier** extracts a numeric final answer from the raw completion and scores it against the oracle answer within a per-task tolerance.
- **Failure tagger** attaches each scored result to its task’s failure family, enabling aggregation by mechanism rather than only by task or model.
- **Statistical reporting** computes pass rates with Wilson score confidence intervals, appropriate for the small-sample binomial proportions typical of a hand-constructed adversarial task set, and renders bar-chart figures directly from scored results.

5 Pilot Results

We report results on a two-task pilot set spanning ASC 715 (pension accounting) and ASC 830 (foreign currency consolidation), instantiating the temporal coupling and assumption violation mechanisms respectively. Both tasks were validated per Section 3: the oracle solution passes within a 1% tolerance and the naive reflex solution fails by a wide margin (24.3% and 20.0% relative error respectively). Each task is accompanied by multiple source artifacts (a SQLite data extract, and either a spreadsheet or a short policy memo PDF) mirroring the multi-source structure of the source platform’s tasks, fictionalized to avoid disclosing any live evaluation content.

Table 1: Pilot task summary.

Task	Domain	Failure mechanism	Naive/oracle gap
demo_pension_restatement	ASC 715	Temporal coupling	24.3%
demo_fx_consolidation	ASC 830	Assumption violation	20.0%

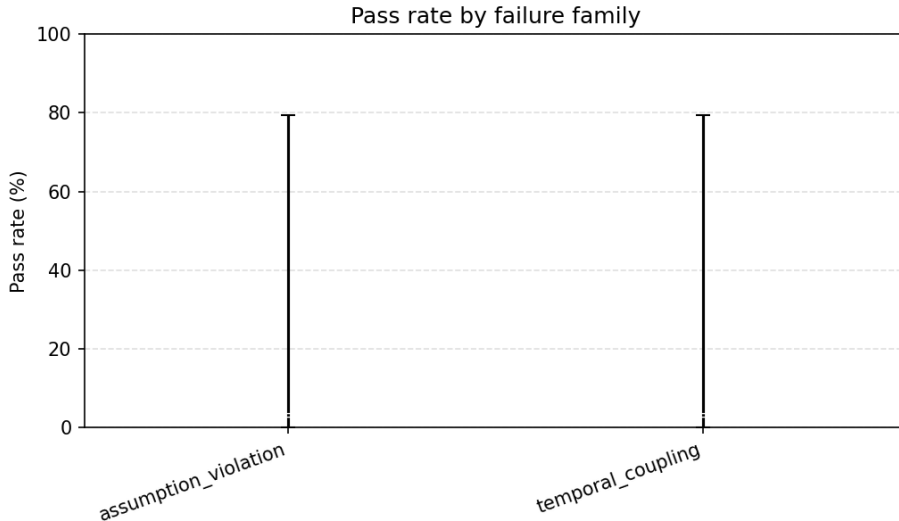


Figure 1: Pass rate by failure family (pilot run; replace with results from a full model sweep before submission).

Note: the figures and tables in this draft are generated from a harness smoke test using a deterministic offline mock client, to confirm the pipeline runs end to end. Before submission, re-run `src/run_pipeline.py` with `--provider anthropic` and `--provider openai` (and any additional providers) against the full task set to produce real results.

6 Discussion and Limitations

The pilot set reported here is intentionally small; the central contribution of this paper is the task construction methodology and harness, not a large-scale leaderboard. A full submission should expand the task set substantially (target: 30–50 tasks spanning multiple failure families and domains) and report results across a representative set of current frontier models. We also note that automatic numeric answer extraction is a simplification; tasks requiring structured or free-text answers

would need a more general verifier, and any use of a model-based judge for non-numeric answers should be cross-checked against known LLM-judge reliability issues.

The current harness evaluates models via single-turn text completion: each task’s source artifacts (data extracts, spreadsheets, policy documents) are flattened into the instruction text rather than provided as files a tool-using agent must open and parse. This keeps the harness simple and provider-agnostic, but it understates the difficulty of the source platform’s actual tasks, which are agentic and require an agent to independently locate, parse, and cross-reference multiple raw files. Extending the harness to an agentic, tool-using evaluation loop (e.g. giving the model file-system and code-execution access rather than a single flattened prompt) is the highest-value next step for making pilot results comparable to the source platform’s own difficulty gate.

7 Conclusion

We present a methodology and reproducible harness for constructing adversarial evaluation tasks that isolate silent-attractor reasoning failures in frontier language models, together with a nine-family taxonomy of the mechanisms by which such failures are constructed. We release a pilot task set and invite extension to additional domains and failure families.